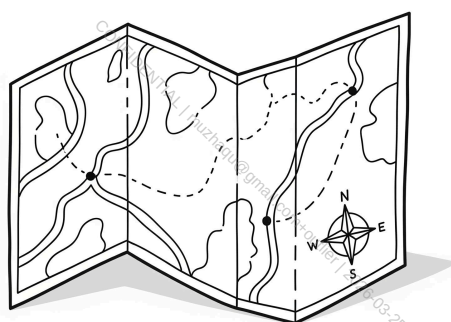




Menu Territory

Common Errors and Golden Examples

Change Log

 Date	 Updates	 Notes/Images/Links
Mar 25, 2026	→ "Unified Diff Accuracy Issue" Common Error updated	
Mar 17, 2026	→ Initial version. → Example #1 → Example #2 → Example #3	

✗ Common Errors to Avoid

1. ✗ Accurate Unified Diff is not provided as reference text in the prompt section.

The unified diff (**provided in the prompt section as reference text**) must be accurate and should come from the repo link using base and target commits using the format below:

Format: **repo link/compare/<base commit>...<target commit>.diff**

Example:

<https://github.com/hyprwm/Hyprland/compare/eb3b38d40baca5c05ddbc1507b3d3f02a0ccb164...803dee81c17afda1bcef147edd003f0058c803d9.diff>

Using any other diffs which do not match the diff from repo link is considered as inaccurate and could cause the bad task (prompt, response and rubrics as bad).

2. ❌ Rubrics are not self-contained.

Some rubrics require referencing the original prompt to evaluate the quality of the model's response. However, during evaluation, we only have access to the model's response. Rubrics must therefore be fully self-contained and allow assessment based solely on the response itself. Also, "Valid" / "proper" is not self-contained. We should always try to avoid those vague words.

❌ The Bad Example

None

The response verifies that proper feedback is given by specifying what to change, and why, without hallucinating files that are not in the diff.

Why this fails:

When evaluating the model using this criterion, we do not have access to the diff, only the model's response. Imagine that you only have access to the model response and are trying to evaluate if it fulfilled the rubric item. Will you be able to evaluate accurately without referencing anything else?

3. ❌ Rubrics are not atomic.

Each rubric should evaluate one thing only — no bundling of multiple behaviors. Ask yourself if the criterion is evaluating more than one idea. A rubric should not contain two complete sentences under any circumstances or overlap with others.

4. ❌ Blockers are based on assumptions rather than concrete code behavior.

A blocker should be grounded in the actual implementation of the provided Diff file and observable code logic in the Diff, not on speculative or hypothetical scenarios.

5. ❌ Fix-related rubrics are overly restrictive.

Some rubrics exclude alternative valid solutions by prescribing a single specific fix. To maintain flexibility and accommodate multiple correct implementation approaches, please use the following format:

"The response suggests a fix for <blocker_x>, for example <suggested_fix>."

❌ The Bad Example

None

The response suggests removing the inspector widget from the splitter before calling ``deleteLater()`` in the ``set_position()`` method in ``qutebrowser/browser/inspector.py``.

Why this fails:

The overfitting criterion for suggested fixes for the blockers is too restrictive - it allows only one valid fix suggestion instead of allowing for multiple solutions. Any method that ensures the widget is detached from the splitter before deletion is valid

✓ The Good Example

None

The response suggests a fix for Blocker: the Inspector widget is not removed from splitter when closed, for example, removing the inspector widget from the splitter before calling ``deleteLater()`` in the ``set_position()`` method in ``qutebrowser/browser/inspector.py``.

6. ✗ Rubrics cannot be satisfied with the provided Golden Response.

All rubrics must be passable by the Golden Response. Please ensure that the Golden Response represents the optimal solution. Even "nice-to-have" rubrics should be satisfied by it.

7. ✗ The average score of the three models is above 70%

Make sure that the average score of the three models is **below 70%**. If the average score is above or equal to 70%, the task will automatically fail.

8. Every task must include at least one blocker issue and non-blocking suggestions, along with the corresponding rubric. If there aren't any blocking issues in the provided diff, please report the task ID in the war room and skip it.

9. There must be at least one rubric related to non-blocking suggestions. We recommend using formats like:

"The response suggests non-blocking issues such as XXX."

"The response includes a non-blocking suggestion that XXX."

10. Consistency Check: If your prompt mentions specific bugs or the need for particular tests within the provided diff, ensure the golden response fully resolves or addresses those specific points.

✓ Good Task Examples

▲ Please do not copy and paste ANY information from the tasks.

▲ Use them only as a reference.

Example #1

None

Task Metadata:

Github Repo

<https://github.com/keycloak/keycloak>

Repository Name and Description

keycloak

Keycloak: Open Source Identity and Access Management For Modern Applications and Services

Base Commit

8af4a5fc852b2aeba8617ef71cf86573fe28b27f

Target Commit

d0a93726882a35f76024dcdaeff9ef67f9081eff

PR Link

<https://github.com/keycloak/keycloak/pull/7035>

Issue Title

[KEYCLOAK-14046] Include groups in user creation via Admin Console

Diff:

```
diff --git a/themes/src/main/resources/theme/base/admin/messages/admin-messages_en.properties
b/themes/src/main/resources/theme/base/admin/messages/admin-messages_en.properties
index 18e913f724..c7d073858e 100644
--- a/themes/src/main/resources/theme/base/admin/messages/admin-messages_en.properties
+++ b/themes/src/main/resources/theme/base/admin/messages/admin-messages_en.properties
@@ -1394,6 +1394,11 @@ unlock-user=Unlock user
    federation-link=Federation Link
    email-verified=Email Verified
    email-verified.tooltip=Has the user's email been verified?
+groups-joining=Groups
+groups-joining.tooltip=Groups the user will be joining. To add a group, search for any existing one
and select it.
+groups-joining-select.placeholder=Select existing group
+groups-joining-no-selected=No group selected
+groups-joining-path=Path
    required-user-actions=Required User Actions
    required-user-actions.tooltip=Require an action when the user logs in. 'Verify email' sends an email
to the user to verify their email address. 'Update profile' requires user to enter in new personal
information. 'Update password' requires user to enter in a new password. 'Configure OTP' requires
setup of a mobile password generator.
    locale=Locale
diff --git a/themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js
b/themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js
index 7d7e4d0d3e..34df0290c3 100755
--- a/themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js
+++ b/themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js
@@ -385,14 +385,15 @@ module.controller('UserDetailCtrl', function($scope, realm, user, BruteForceUser
    Components,
    UserImpersonation, RequiredActions,
    UserStorageOperations,
-    $location, $http, Dialog, Notifications, $translate) {
+    $location, $http, Dialog, Notifications, $translate,
+    Groups) {
    $scope.realm = realm;
    $scope.create = !user.id;
    $scope.editUsername = $scope.create || $scope.realm.editUsernameAllowed;
    $scope.emailAsUsername = $scope.realm.registrationEmailAsUsername;
+    $scope.groupSearch = { selectedGroup : null };

    if ($scope.create) {
-        $scope.user = { enabled: true, attributes: {} }
+        $scope.user = { enabled: true, attributes: {}, groups: [] }
```

```

    } else {
        if (!user.attributes) {
            user.attributes = {}
@@ -464,6 +465,8 @@ module.controller('UserDetailCtrl', function($scope, realm, user, BruteForceUser
            convertAttributeValuesToLists());

            if ($scope.create) {
+                pushSelectedGroupsToUser();
+
                User.save({
                    realm: realm.realm
                }, $scope.user, function (data, headers) {
@@ -513,6 +516,51 @@ module.controller('UserDetailCtrl', function($scope, realm, user, BruteForceUser
                }
            }

+            function pushSelectedGroupsToUser() {
+                var groups = $scope.user.groups;
+                if ($scope.selectedGroups) {
+                    for (i = 0; i < $scope.selectedGroups.length; i++) {
+                        var groupPath = $scope.selectedGroups[i].path;
+                        if (!groups.includes(groupPath)) {
+                            groups.push(groupPath);
+                        }
+                    }
+                }
+            }
+
+            function bfs(tree, collection) {
+                if (!tree["subGroups"] || tree["subGroups"].length === 0) return;
+                for (var i=0; i < tree["subGroups"].length; i++) {
+                    var child = tree["subGroups"][i]
+                    collection.push(child);
+                    bfs(child, collection);
+                }
+                return;
+            }
+
+            function flattenGroups(groups) {
+                var flattenedGroups = [];
+                if (!groups || groups.length === 0) return groups;
+                for (var i=0; i < groups.length; i++) {
+                    flattenedGroups.push(groups[i]);
+                    bfs(groups[i], flattenedGroups);

```

```

+     }
+
+     return flattenedGroups;
+   }
+
+   /**
+    * Only keep groups that :
+    * - include the search term in their path
+    * - are not already selected
+    */
+   function filterSearchedGroups(groups, term, selectedGroups) {
+     if (!groups || groups.length === 0) return groups;
+     if (!selectedGroups) selectedGroups = [];
+
+     return groups.filter(group => group.path?.includes(term) && !selectedGroups.some(selGroup =>
+ selGroup.id === group.id));
+   }
+
+   $scope.reset = function() {
+     $scope.user = angular.copy(user);
+     $scope.changed = false;
+
+@@ -530,6 +578,67 @@ module.controller('UserDetailCtrl', function($scope, realm, user, BruteForceUser
+     $scope.removeAttribute = function(key) {
+       delete $scope.user.attributes[key];
+     }
+
+     $scope.groupsUiSelect = {
+       minimumInputLength: 1,
+       delay: 500,
+       allowClear: true,
+       query: function (query) {
+         var data = {results: []};
+         if ('' !== query.term.trim()) {
+           query.callback(data);
+           return;
+         }
+
+         $scope.query = {
+           realm: realm.realm,
+           search: query.term.trim(),
+           max : 20,
+           first : 0
+         };
+
+         Groups.query($scope.query, function(response) {
+           data.results = filterSearchedGroups(flattenGroups(response), query.term.trim(),
+ $scope.selectedGroups);

```

```

+         query.callback(data);
+     });
+ },
+     formatResult: function(object, container, query) {
+         object.text = object.path;
+         return object.path;
+     }
+ };
+
+ $scope.removeGroup = function(list, group) {
+     for (i = 0; i < angular.copy(list).length; i++) {
+         if (group.id == list[i].id) {
+             list.splice(i, 1);
+         }
+     }
+ }
+
+ $scope.selectGroup = function(group) {
+     if (!group || !group.id) {
+         return;
+     }
+
+     $scope.groupSearch.selectedGroup = group;
+
+     if (!$scope.selectedGroups) {
+         $scope.selectedGroups = [];
+     }
+
+     for (i = 0; i < $scope.selectedGroups.length; i++) {
+         if ($scope.selectedGroups[i].id == group.id) {
+             return;
+         }
+     }
+
+     $scope.selectedGroups.push(group);
+     $scope.groupSearch.selectedGroup = null;
+ }
+
+ $scope.clearGroupSelection = function() {
+     $scope.groupSearch.selectedGroup = null;
+     $('#groups').val(null).trigger('change.select2');
+ }
+ });

```

```
module.controller('UserCredentialsCtrl', function($scope, realm, user, $route, $location,
RequiredActions, User, UserExecuteActionsEmail,
diff --git a/themes/src/main/resources/theme/base/admin/resources/partials/group-list.html
b/themes/src/main/resources/theme/base/admin/resources/partials/group-list.html
index 731e28d42c..3412e6a2d9 100755
--- a/themes/src/main/resources/theme/base/admin/resources/partials/group-list.html
+++ b/themes/src/main/resources/theme/base/admin/resources/partials/group-list.html
@@ -1,7 +1,7 @@
<div class="col-sm-9 col-md-10 col-sm-push-3 col-md-push-2">
    <kc-tabs-group-list></kc-tabs-group-list>
```

```
- <table class="table table-striped table-bordered" style="margin-bottom: 0">
+ <table class="table table-striped table-bordered" id="group-table" style="margin-bottom: 0">
    <thead>
    <tr>
        <th class="kc-table-actions" colspan="5">
```

```
diff --git a/themes/src/main/resources/theme/base/admin/resources/partials/user-detail.html
b/themes/src/main/resources/theme/base/admin/resources/partials/user-detail.html
index 31583774fa..d7b4623886 100755
```

```
--- a/themes/src/main/resources/theme/base/admin/resources/partials/user-detail.html
+++ b/themes/src/main/resources/theme/base/admin/resources/partials/user-detail.html
@@ -97,6 +97,42 @@
        </div>
        <kc-tooltip>{{:: 'email-verified.tooltip' | translate}}</kc-tooltip>
    </div>
```

```
+ <div class="form-group clearfix block" data-ng-if="create">
+     <label class="col-md-2 control-label" for="selected-groups">{{:: 'groups-joining' |
+ translate}}</label>
+     <div class="col-sm-6">
+         <table class="table table-striped table-bordered" style="margin-top: 0px"
+ id="selected-groups">
+             <thead>
+             <tr>
+                 <th class="kc-table-actions" colspan="2">
+                     <div class="form-inline col-md-12 select2-container-single">
+                         <div class="form-group">
+                             <div class="input-group">
+                                 <input type="hidden" ui-select2="groupsUiSelect"
+ id="groups" data-ng-change="selectGroup(groupSearch.selectedGroup);"
+ data-ng-model="groupSearch.selectedGroup" data-placeholder="{{:: 'groups-joining-select.placeholder' |
+ translate}}..."/>
+                             </div>
+                         </div>
+                     </div>
+                 </th>
+             </tr>
```

```

+         <tr data-ng-hide="!selectedGroups || selectedGroups.length == 0">
+             <th>{{:: 'groups-joining-path' | translate}}</th>
+             <th width="20%">{{:: 'actions' | translate}}</th>
+         </tr>
+     </thead>
+     <tbody>
+         <tr ng-repeat="group in selectedGroups">
+             <td>{{group.path}}</td>
+             <td class="kc-action-cell" ng-click="removeGroup(selectedGroups, group);"
+ style="vertical-align: middle">{{:: 'remove' | translate}}
+         </td>
+     </tr>
+     <tr data-ng-show="!selectedGroups || selectedGroups.length == 0">
+         <td class="text-muted" colspan="2">{{:: 'groups-joining-no-selected' |
+ translate}}</td>
+     </tr>
+ </tbody>
+ </table>
+ </div>
+ <kc-tooltip>{{:: 'groups-joining.tooltip' | translate}}</kc-tooltip>
+ </div>
+ <div class="form-group clearfix">
+     <label class="col-md-2 control-label" for="reqActions">{{:: 'required-user-actions' |
+ translate}}</label>

```

```

diff --git a/themes/src/main/resources/theme/keycloak/admin/resources/css/styles.css
b/themes/src/main/resources/theme/keycloak/admin/resources/css/styles.css
index 681b3fc045..5fedbddd5 100755

```

```

--- a/themes/src/main/resources/theme/keycloak/admin/resources/css/styles.css
+++ b/themes/src/main/resources/theme/keycloak/admin/resources/css/styles.css
@@ -190,6 +190,20 @@ th.w-40 {
     height: 26px;
 }

```

```

+.select2-container-single {
+    padding: 0;
+    margin: 0;
+}
+
+.select2-container-single .form-group {
+    width: 100%;
+    margin: 0;
+}
+

```

```
+ .select2-container-single .form-group .input-group {  
+   width: 100%;  
+ }  
+  
+ /***** html select *****/  
+ .overflow-select {  
+   overflow: auto;
```

User Prompt:

You are a senior software engineer reviewing a Pull Request diff in the Keycloak repository. Keycloak is an open-source Identity and Access Management platform that provides authentication, user federation, authorization, and administrative tooling. The Admin Console is a critical interface for managing users, roles, and groups, so UI changes must remain consistent with the underlying REST API behavior and not introduce security or state inconsistencies.

Your role is to verify that the changes correctly solve the stated problem, are production-ready, and do not introduce any bugs or issues.

Keycloak's POST /users Admin REST API now supports specifying group membership during user creation. The goal of this PR is to update the Admin Console UI to allow administrators to assign groups when creating a new user, aligning the UI capabilities with the API. This involves adding a new field in the user creation form to select groups and wiring it correctly into the user creation request payload, as well as adding the necessary localized messages.

I provided the diff as a reference text below. Provide the detailed diff review with the following structure:

Summary

Blockers (for each blocker, include what is wrong, why it matters, how to fix it and where it exists in the diff)

Non-blocking suggestions

Tests/Validation

Golden response:

1. Summary

This patch adds a group selection field to the Admin Console user creation form, allowing administrators to assign groups at the time a new user is created. The overall approach is correct and aligns the UI with the existing

- POST /users` API capability. However, there are two issues that must be addressed before merge: the Select2 input widget is not visually reset after a group is selected (causing a clear UX regression), and the loop counter variables are implicitly global across three functions.

2. Blockers

- Blocker 1: Select2 input does not reset after group selection.

Why it matters:

`\$scope.selectGroup()` sets `\$scope.groupSearch.selectedGroup = null` after pushing the group to `\$scope.selectedGroups`. This updates the Angular model, but Select2 maintains its own internal DOM state independently. Setting the angular model to `null` does not cause Select2 to visually clear its input field. As a result, the selected group's name remains displayed in the search box even after it has been added to the table, making the UI appear broken to the admin.

Why it matters:

A `\$scope.clearGroupSelection()` helper already exists in the same controller for exactly this purpose: it resets both the Angular model and fires the necessary Select2 event (`\$('#groups').val(null).trigger('change.select2')`), but it is never called from `\$scope.selectGroup()`. This is a functional UX regression that is visible on every group selection.

Exact fix:

At the end of `\$scope.selectGroup()`, replace the direct assignment `\$scope.groupSearch.selectedGroup = null` with a call to the existing `\$scope.clearGroupSelection()` helper instead. This ensures both the Angular model and the Select2 widget state are reset together.

Where:

The `\$scope.selectGroup` function in the
`themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js` file.

- Blocker 2: Implicit global variable `i` in three functions.

What's wrong:

The loop counter `i` is used without `var` in `pushSelectedGroupsToUser()`, `$scope.removeGroup()`, and `$scope.selectGroup()`. In non-strict JavaScript, an undeclared variable assignment creates a property on the global `window` object.

Why it matters:

All three functions share the same `global.i`. If any two of these functions are active at the same time (for example, if Angular's digest cycle triggers `selectGroup` while `removeGroup` is mid-loop, or if `pushSelectedGroupsToUser` is called during form submission while a concurrent watch resolves), they will overwrite each other's loop counter. This leads to incorrect iteration: elements being skipped, iterated multiple times, or array access going out of bounds. This creates a real bug, and is not just a style concern.

Exact fix:

Declare `var i` explicitly in each of the three for-loop initializers: in `pushSelectedGroupsToUser()`, `$scope.removeGroup()`, and `$scope.selectGroup()`. This scopes the variable to the function rather than the global object.

Where:

`themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js`.

3. Non-blocking suggestions

- `Groups.query()` has no error callback: if the API call fails, the search silently returns no results with no user feedback. Add an error callback that surfaces a notification to the user.

4. Tests/Validation

The following tests should be performed to ensure that the patch works as expected:

- Select2 reset: select a group from the dropdown. Before the fix, the input remains populated with the selected group's name after it is added to the table. After the fix, the input clears back to the placeholder immediately.
- Duplicate prevention: attempt to select the same group twice. The selected groups table must not contain duplicate entries on either attempt.

- Whitespace only search input: Type only spaces in the search field. No API call should be triggered and the dropdown should not open.

Rubric:

Criteria	Rubric Type	Rubric Weights Mandatory = 5 Important = 3 Nice to Have = 1
The response follows a structure with Summary, Blockers, Non-blocking Suggestions and Tests/Validations sections.	Instruction following	mandatory
The response identifies that <code>\$scope.selectGroup()</code> in <code>themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js</code> does not visually reset the Select 2 input widget after a group is added to the selection list.	bug discovery	mandatory
The response suggests a fix for the Select2 widget not being reset, for example, calling <code>\$scope.clearGroupSelection()</code> inside <code>themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js</code> instead of only setting <code>\$scope.groupSearch.selectedGroup = null</code> .	bug discovery	mandatory
The response identifies that the loop variable <code>i</code> is used without a <code>var</code> declaration in the <code>pushSelectedGroupsToUser()</code> function in <code>themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js</code> , making it an implicit global variable that can be	bug discovery	mandatory

overwritten by any other function using an undeclared <code>i</code> concurrently.		
The response identifies that the loop variable <code>i</code> is used without a <code>var</code> declaration in the <code>\$scope.removeGroup()</code> function in <code>themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js</code> , making it an implicit global that can be overwritten by any other function using an undeclared <code>i</code> concurrently.	bug discovery	mandatory
The response identifies that the loop variable <code>i</code> is used without a <code>var</code> declaration in the <code>\$scope.selectGroup()</code> function in <code>themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js</code> , making it an implicit global variable that can be overwritten by any other function using an undeclared <code>i</code> concurrently.	bug discovery	mandatory
The response recommends a fix for declaration issue, for example, declaring <code>var i</code> explicitly in the for-loop initializer of the <code>pushSelectedGroupsToUser()</code> function in <code>themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js</code> .	bug discovery	mandatory
The response recommends a fix for declaration issue, for example, declaring <code>var i</code> explicitly in the for-loop initializer of the <code>\$scope.removeGroup()</code> function in <code>themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js</code> .	bug discovery	mandatory
The response recommends a fix for declaration issue, for example, declaring <code>var i</code> explicitly in the for-loop initializer of the <code>\$scope.selectGroup()</code> function in <code>themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js</code> .	bug discovery	mandatory

The response recommends performing a test to verify that the Select2 input clears back to the placeholder after a group is selected and added to the list.	instruction following	important
The response includes a non-blocking issue that the <code>Groups.query()</code> call inside the <code>groupsUiSelect</code> query function in <code>themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js</code> has no error callback, causing API failures to produce no user visible feedback.	bug discovery	important
The response recommends performing a test that verifies that attempting to add the same group twice does not result in duplicate entries in the selected group list.	instruction following	important
The response recommends performing a test that verifies that entering only whitespace in the group search field does not trigger an API call.	instruction following	important

Example #2

None

Task Metadata:

Github Repo

<https://github.com/colmap/colmap>

Repository Name and Description

colmap/colmap

COLMAP - Structure-from-Motion and Multi-View Stereo

Base Commit

84302728b830798d37256f2896f7355879200e5a

Target Commit

562715a11f9dee28d5f23062082295889a5b925d

PR Link

<https://github.com/colmap/colmap/pull/1148>

Issue Title

Parallelize stereo fusion; needs pre-loading of entire workspace

Diff:

```
diff --git a/src/exe/colmap.cc b/src/exe/colmap.cc
index 82968e77..202a30c9 100644
--- a/src/exe/colmap.cc
+++ b/src/exe/colmap.cc
@@ -44,6 +44,7 @@
#include "feature/extraction.h"
#include "feature/matching.h"
#include "feature/utils.h"
+ #include "mvs/fusion.h"
#include "mvs/meshing.h"
#include "mvs/patch_match.h"
#include "retrieval/visual_index.h"
@@ -267,6 +268,7 @@ int RunStereoFuser(int argc, char** argv) {
    std::string pmvs_option_name = "option-all";
    std::string output_type = "PLY";
    std::string output_path;
+   std::string bbox_path;

    OptionManager options;
    options.AddRequiredOption("workspace_path", &workspace_path);
@@ -278,6 +280,7 @@ int RunStereoFuser(int argc, char** argv) {
    options.AddDefaultOption("output_type", &output_type,
                             "{BIN, TXT, PLY}");
    options.AddRequiredOption("output_path", &output_path);
+   options.AddDefaultOption("bbox_path", &bbox_path);
    options.AddStereoFusionOptions();
    options.Parse(argc, argv);

@@ -297,6 +300,20 @@ int RunStereoFuser(int argc, char** argv) {
    return EXIT_FAILURE;
}
```

```

    }

+   if (!bbox_path.empty()) {
+       std::ifstream file(bbox_path);
+       if (file.is_open()) {
+           auto& min_bound = options.stereo_fusion->bounds.first;
+           auto& max_bound = options.stereo_fusion->bounds.second;
+           file >> min_bound(0) >> min_bound(1) >> min_bound(2);
+           file >> max_bound(0) >> max_bound(1) >> max_bound(2);
+           file.close();
+       } else {
+           std::cout << "WARN: Invalid bounds path: \"" << bbox_path
+               << "\" - continuing without bounds check" << std::endl;
+       }
+   }

+   mvs::StereoFusion fuser(*options.stereo_fusion, workspace_path,
+                           workspace_format, pmvs_option_name, input_type);

diff --git a/src/mvs/fusion.cc b/src/mvs/fusion.cc
index 8c5d1180..22c920b1^100755
--- a/src/mvs/fusion.cc
+++ b/src/mvs/fusion.cc
@@ -83,6 +83,7 @@ int FindNextImage(const std::vector<std::vector<int>>& overlapping_images,
void StereoFusionOptions::Print() const {
#define PrintOption(option) std::cout << #option ": " << option << std::endl
PrintHeading2("StereoFusion::Options");
+   PrintOption(mask_path);
+   PrintOption(max_image_size);
+   PrintOption(min_num_pixels);
+   PrintOption(max_num_pixels);
@@ -91,7 +92,12 @@ void StereoFusionOptions::Print() const {
+   PrintOption(max_depth_error);
+   PrintOption(max_normal_error);
+   PrintOption(check_num_images);
+   PrintOption(use_cache);
+   PrintOption(cache_size);
+   const auto& min_bound = bounds.first.transpose().eval();
+   const auto& max_bound = bounds.second.transpose().eval();
+   PrintOption(min_bound);
+   PrintOption(max_bound);
#undef PrintOption
}

```

```

@@ -151,12 +157,27 @@ void StereoFusion::Run() {
}

    workspace_options.max_image_size = options_.max_image_size;
+ // Saving calculated maps only when the option is set (useful for debugging)
+ // or when using cache to avoid potential multiple calculations of the same
+ // map.
+ workspace_options.save_calculated_maps =
+     options_.save_calculated_maps || options_.use_cache;
    workspace_options.image_as_rgb = true;
    workspace_options.cache_size = options_.cache_size;
    workspace_options.workspace_path = workspace_path_;
    workspace_options.workspace_format = workspace_format_;
    workspace_options.input_type = input_type_;

+ const auto image_names = ReadTextFileLines(JoinPaths(
+     workspace_path_, workspace_options.stereo_folder, "fusion.cfg"));
+ int num_threads = 1;
+ if (options_.use_cache) {
+     workspace_.reset(new Workspace(workspace_options));
+ } else {
+     workspace_.reset(new NoCacheWorkspace(workspace_options));
+     workspace_>Load(image_names);
+     num_threads = omp_get_max_threads();
+ }
    workspace_.reset(new Workspace(workspace_options));

    if (IsStopped()) {
@@ -176,6 +197,18 @@ void StereoFusion::Run() {
        overlapping_images_ = model.GetMaxOverlappingImagesFromPMVS();
    }

+ fusion_queue_.resize(num_threads);
+ fused_point_x_.resize(num_threads);
+ fused_point_y_.resize(num_threads);
+ fused_point_z_.resize(num_threads);
+ fused_point_nx_.resize(num_threads);
+ fused_point_ny_.resize(num_threads);
+ fused_point_nz_.resize(num_threads);
+ fused_point_r_.resize(num_threads);
+ fused_point_g_.resize(num_threads);
+ fused_point_b_.resize(num_threads);
+ fused_point_visibility_.resize(num_threads);
+

```

```

used_images_.resize(model.images.size(), false);
fused_images_.resize(model.images.size(), false);
fused_pixel_masks_.resize(model.images.size());
@@ -185,8 +218,6 @@ void StereoFusion::Run() {
    inv_P_.resize(model.images.size());
    inv_R_.resize(model.images.size());

-   const auto image_names_ = ReadTextFileLines(JoinPaths(
-       workspace_path_, workspace_options.stereo_folder, "fusion.cfg"));
    for (const auto& image_name : image_names) {
        const int image_idx = model.GetImageIdx(image_name);

@@ -206,9 +237,7 @@ void StereoFusion::Run() {
    used_images_.at(image_idx) = true;

-   fused_pixel_masks_.at(image_idx) =
-       Mat<bool>(depth_map.GetWidth(), depth_map.GetHeight(), 1);
-   fused_pixel_masks_.at(image_idx).Fill(false);
+   InitFusedPixelMask(image_idx, depth_map.GetWidth(), depth_map.GetHeight());

    depth_map_sizes_.at(image_idx) =
        std::make_pair(depth_map.GetWidth(), depth_map.GetHeight());
@@ -235,6 +264,8 @@ void StereoFusion::Run() {
    .transpose();
}

+   std::cout << "Starting fusion with " << num_threads << " threads"
+   << std::endl;
    size_t num_fused_images = 0;
    for (int image_idx = 0; image_idx >= 0;
        image_idx = internal::FindNextImage(overlapping_images_, used_images_,
@@ -246,27 +277,30 @@ void StereoFusion::Run() {
        Timer timer;
        timer.Start();

-       std::cout << StringPrintf("Fusing image [%d/%d]", num_fused_images + 1,
-                               model.images.size())
-       std::cout << StringPrintf("Fusing image [%d/%d] with index %d",
+       std::cout << StringPrintf("Fusing image [%d/%d] with index %d",
+                               num_fused_images + 1, model.images.size(),
+                               image_idx)
        << std::flush;

        const int width = depth_map_sizes_.at(image_idx).first;

```

```

const int height = depth_map_sizes_.at(image_idx).second;
const auto& fused_pixel_mask = fused_pixel_masks_.at(image_idx);

-   FusionData data;
-   data.image_idx = image_idx;
-   data.traversal_depth = 0;
-
-   for (data.row = 0; data.row < height; ++data.row) {
-       for (data.col = 0; data.col < width; ++data.col) {
-           if (fused_pixel_mask.Get(data.row, data.col)) {
+ #pragma omp parallel for schedule(dynamic) num_threads(num_threads)
+   for (int row = 0; row < height; ++row) {
+       for (int col = 0; col < width; ++col) {
+           if (fused_pixel_mask.Get(row, col)) {
+               continue;
+           }
-       fusion_queue_.push_back(data);
-
-       Fuse();
+       FusionData data;
+       data.image_idx = image_idx;
+       data.traversal_depth = 0;
+       data.row = row;
+       data.col = col;
+
+       const int tid = omp_get_thread_num();
+       fusion_queue_[tid].push_back(std::move(data));
+       Fuse(tid);
+   }
+ }

```

```

@@ -292,31 +326,54 @@ void StereoFusion::Run() {
    GetTimer().PrintMinutes();
}

```

```

-void StereoFusion::Fuse() {
-   CHECK_EQ(fusion_queue_.size(), 1);
+void StereoFusion::InitFusedPixelMask(int image_idx, size_t width,
+                                       size_t height) {
+   Bitmap mask;
+   Mat<bool>& fused_pixel_mask = fused_pixel_masks_.at(image_idx);
+   const std::string mask_path =
+       JoinPaths(options_.mask_path,

```

```

+         workspace_>GetModel().GetImageName(image_idx) + ".png");
+ fused_pixel_mask = Mat<bool>(width, height, 1);
+ if (!options_.mask_path.empty() && ExistsFile(mask_path) &&
+     mask.Read(mask_path, false)) {
+     BitmapColor<uint8_t> color;
+     mask.Rescale((int)width, (int)height, FILTER_BOX);
+     for (int row = 0; row < height; ++row) {
+         for (int col = 0; col < width; ++col) {
+             mask.GetPixel(col, row, &color);
+             fused_pixel_mask.Set(row, col, color.r == 0);
+         }
+     }
+ } else {
+     fused_pixel_mask.Fill(false);
+ }
+
+void StereoFusion::Fuse(int tid) {
+    CHECK_EQ(fusion_queue_[tid].size(), 1);

+    Eigen::Vector4f fused_ref_point = Eigen::Vector4f::Zero();
+    Eigen::Vector3f fused_ref_normal = Eigen::Vector3f::Zero();

+    - fused_point_x_.clear();
+    - fused_point_y_.clear();
+    - fused_point_z_.clear();
+    - fused_point_nx_.clear();
+    - fused_point_ny_.clear();
+    - fused_point_nz_.clear();
+    - fused_point_r_.clear();
+    - fused_point_g_.clear();
+    - fused_point_b_.clear();
+    - fused_point_visibility_.clear();
+    -
+    - while (!fusion_queue_.empty()) {
+        - const auto data = fusion_queue_.back();
+    + fused_point_x_[tid].clear();
+    + fused_point_y_[tid].clear();
+    + fused_point_z_[tid].clear();
+    + fused_point_nx_[tid].clear();
+    + fused_point_ny_[tid].clear();
+    + fused_point_nz_[tid].clear();
+    + fused_point_r_[tid].clear();
+    + fused_point_g_[tid].clear();

```

```

+ fused_point_b[tid].clear();
+ fused_point_visibility[tid].clear();
+
+ while (!fusion_queue[tid].empty()) {
+     const auto data = fusion_queue[tid].back();
+     const int image_idx = data.image_idx;
+     const int row = data.row;
+     const int col = data.col;
+     const int traversal_depth = data.traversal_depth;

-     fusion_queue_.pop_back();
+     fusion_queue_[tid].pop_back();

    // Check if pixel already fused.
    auto& fused_pixel_mask = fused_pixel_masks_.at(image_idx);
@@ -383,17 +440,27 @@ void StereoFusion::Fuse() {
    // Set the current pixel as visited.
    fused_pixel_mask.Set(row, col, true);

+
+     // Pixels out of bounds are filtered
+     if (xyz(0) < options_.bounds.first(0) ||
+         xyz(1) < options_.bounds.first(1) ||
+         xyz(2) < options_.bounds.first(2) ||
+         xyz(0) > options_.bounds.second(0) ||
+         xyz(1) > options_.bounds.second(1) ||
+         xyz(2) > options_.bounds.second(2)) {
+         continue;
+     }
+
    // Accumulate statistics for fused point.
-     fused_point_x_.push_back(xyz(0));
-     fused_point_y_.push_back(xyz(1));
-     fused_point_z_.push_back(xyz(2));
-     fused_point_nx_.push_back(normal(0));
-     fused_point_ny_.push_back(normal(1));
-     fused_point_nz_.push_back(normal(2));
-     fused_point_r_.push_back(color.r);
-     fused_point_g_.push_back(color.g);
-     fused_point_b_.push_back(color.b);
-     fused_point_visibility_.insert(image_idx);
+     fused_point_x_[tid].push_back(xyz(0));
+     fused_point_y_[tid].push_back(xyz(1));
+     fused_point_z_[tid].push_back(xyz(2));
+     fused_point_nx_[tid].push_back(normal(0));

```

```

+   fused_point_ny_[tid].push_back(normal(1));
+   fused_point_nz_[tid].push_back(normal(2));
+   fused_point_r_[tid].push_back(color.r);
+   fused_point_g_[tid].push_back(color.g);
+   fused_point_b_[tid].push_back(color.b);
+   fused_point_visibility_[tid].insert(image_idx);

    // Remember the first pixel as the reference.
    if (traversal_depth == 0) {
@@ -401,7 +468,7 @@ void StereoFusion::Fuse() {
        fused_ref_normal = normal;
    }

-   if (fused_point_x_.size() >= static_cast<size_t>(options_.max_num_pixels)) {
+   if (fused_point_x_[tid].size() >= static_cast<size_t>(options_.max_num_pixels)) {
        break;
    }

@@ -432,43 +499,43 @@ void StereoFusion::Fuse() {
        continue;
    }

-   fusion_queue_.push_back(next_data);
+   fusion_queue_[tid].push_back(next_data);
    }
}

-   fusion_queue_.clear();
+   fusion_queue_[tid].clear();

-   const size_t num_pixels = fused_point_x_.size();
+   const size_t num_pixels = fused_point_x_[tid].size();
    if (num_pixels >= static_cast<size_t>(options_.min_num_pixels)) {
        PlyPoint fused_point;

        Eigen::Vector3f fused_normal;
-       fused_normal.x() = internal::Median(&fused_point_nx_);
-       fused_normal.y() = internal::Median(&fused_point_ny_);
-       fused_normal.z() = internal::Median(&fused_point_nz_);
+       fused_normal.x() = internal::Median(&fused_point_nx_[tid]);
+       fused_normal.y() = internal::Median(&fused_point_ny_[tid]);
+       fused_normal.z() = internal::Median(&fused_point_nz_[tid]);
        const float fused_normal_norm = fused_normal.norm();
        if (fused_normal_norm < std::numeric_limits<float>::epsilon()) {

```

```

return;

-   fused_point.x = internal::Median(&fused_point_x_);
-   fused_point.y = internal::Median(&fused_point_y_);
-   fused_point.z = internal::Median(&fused_point_z_);
+   fused_point.x = internal::Median(&fused_point_x_[tid]);
+   fused_point.y = internal::Median(&fused_point_y_[tid]);
+   fused_point.z = internal::Median(&fused_point_z_[tid]);

fused_point.nx = fused_normal.x() / fused_normal_norm;
fused_point.ny = fused_normal.y() / fused_normal_norm;
fused_point.nz = fused_normal.z() / fused_normal_norm;

fused_point.r = TruncateCast<float, uint8_t>(
-   std::round(internal::Median(&fused_point_r_)));
+   std::round(internal::Median(&fused_point_r_[tid]));
fused_point.g = TruncateCast<float, uint8_t>(
-   std::round(internal::Median(&fused_point_g_)));
+   std::round(internal::Median(&fused_point_g_[tid]));
fused_point.b = TruncateCast<float, uint8_t>(
-   std::round(internal::Median(&fused_point_b_)));
-
-   fused_points_.push_back(fused_point);
-   fused_points_visibility_.emplace_back(fused_point_visibility_.begin(),
-                                         fused_point_visibility_.end());
+   std::round(internal::Median(&fused_point_b_[tid]));
+
+#pragma omp critical(update_fused_points)
+   {
+   fused_points_.push_back(fused_point);
+   fused_points_visibility_.emplace_back(
+   fused_point_visibility_[tid].begin(),
+   fused_point_visibility_[tid].end());
+   }
+   }
+   }

```

```

diff --git a/src/mvs/fusion.h b/src/mvs/fusion.h
index ddd413d0..f0ad8bbf 100755
--- a/src/mvs/fusion.h
+++ b/src/mvs/fusion.h
@@ -53,6 +53,9 @@ namespace colmap {
     namespace mvs {

```

```

    struct StereoFusionOptions {
+ // Path for PNG masks. Same format expected as ImageReaderOptions
+ std::string mask_path = "";
+
+ // Maximum image size in either dimension.
+ int max_image_size = -1;

@@ -77,6 +80,12 @@ struct StereoFusionOptions {
+ // Number of overlapping images to transitively check for fusing points.
+ int check_num_images = 50;

+ // Flag indicating whether to use LRU cache or pre-load all data
+ bool use_cache = false;
+
+ // Flag to force saving of calculated maps from the workspace
+ bool save_calculated_maps = false;
+
+ // Cache size in gigabytes for fusion. The fusion keeps the bitmaps, depth
+ // maps, normal maps, and consistency graphs of this number of images in
+ // memory. A higher value leads to less disk access and faster fusion, while
@@ -84,6 +93,10 @@ struct StereoFusionOptions {
+ // consume a lot of memory, if the consistency graph is dense.
+ double cache_size = 32.0;

+ std::pair<Eigen::Vector3f, Eigen::Vector3f> bounds =
+ // std::make_pair(Eigen::Vector3f(-FLT_MAX, -FLT_MAX, -FLT_MAX),
+ // Eigen::Vector3f(FLT_MAX, FLT_MAX, FLT_MAX));
+
+ // Check the options for validity.
+ bool Check() const;

@@ -106,7 +119,8 @@ class StereoFusion : public Thread {

private:
+ void Run();
- void Fuse();
+ void InitFusedPixelMask(int image_idx, size_t width, size_t height);
+ void Fuse(int tid);

+ const StereoFusionOptions options_;
+ const std::string workspace_path_;
@@ -120,6 +134,8 @@ class StereoFusion : public Thread {
+ std::vector<char> used_images_;

```

```

std::vector<char> fused_images_;
std::vector<std::vector<int>> overlapping_images_;
+ // Contains image masks of pre-masked and already fused pixels.
+ // Initialized from image masks if provided in StereoFusionOptions.
std::vector<Mat<bool>> fused_pixel_masks_;
std::vector<std::pair<int, int>> depth_map_sizes_;
std::vector<std::pair<float, float>> bitmap_scales_;
@@ -138,23 +154,23 @@ class StereoFusion : public Thread {
};

// Next points to fuse.
- std::vector<FusionData> fusion_queue_;
+ std::vector<std::vector<FusionData>> fusion_queue_;

// Already fused points.
std::vector<PlyPoint> fused_points_;
std::vector<std::vector<int>> fused_points_visibility_;

// Points of different pixels of the currently point to be fused.
- std::vector<float> fused_point_x_;
- std::vector<float> fused_point_y_;
- std::vector<float> fused_point_z_;
- std::vector<float> fused_point_nx_;
- std::vector<float> fused_point_ny_;
- std::vector<float> fused_point_nz_;
- std::vector<uint8_t> fused_point_r_;
- std::vector<uint8_t> fused_point_g_;
- std::vector<uint8_t> fused_point_b_;
- std::unordered_set<int> fused_point_visibility_;
+ std::vector<std::vector<float>> fused_point_x_;
+ std::vector<std::vector<float>> fused_point_y_;
+ std::vector<std::vector<float>> fused_point_z_;
+ std::vector<std::vector<float>> fused_point_nx_;
+ std::vector<std::vector<float>> fused_point_ny_;
+ std::vector<std::vector<float>> fused_point_nz_;
+ std::vector<std::vector<uint8_t>> fused_point_r_;
+ std::vector<std::vector<uint8_t>> fused_point_g_;
+ std::vector<std::vector<uint8_t>> fused_point_b_;
+ std::vector<std::unordered_set<int>> fused_point_visibility_;
};

// Write the visibility information into a binary file of the following format:
diff --git a/src/mvs/workspace.cc b/src/mvs/workspace.cc
index 0e06322a..ad88cdee 100644

```

```

--- a/src/mvs/workspace.cc
+++ b/src/mvs/workspace.cc
@@ -234,5 +234,78 @@ void ImportPMVSWorkspace(const Workspace& workspace,
    }
}

+void NoCacheWorkspace::Load(const std::vector<std::string>& image_names) {
+    const size_t num_images = model_.images.size();
+    bitmaps_.resize(num_images);
+    depth_maps_.resize(num_images);
+    confidence_maps_.resize(num_images);
+    normal_maps_.resize(num_images);
+
+    std::cout << "Loading workspace data..."<< std::endl;
+    #pragma omp parallel for
+    for (int i = 0; i < image_names.size(); i) {
+        const int image_idx = model_.GetImageIdx(image_names[i]);
+
+        if (!HasBitmap(image_idx) || !HasDepthMap(image_idx)) {
+            std::cout
+                << StringPrintf(
+                    "WARNING: Ignoring image %s, because input does not exist.",
+                    image_names[i].c_str())
+                << std::endl;
+            continue;
+        }
+
+        const size_t width = model_.images.at(image_idx).GetWidth();
+        const size_t height = model_.images.at(image_idx).GetHeight();
+
+        // Read and rescale bitmap
+        bitmaps_[image_idx].reset(new Bitmap());
+        bitmaps_[image_idx]->Read(GetBitmapPath(image_idx), options_.image_as_rgb);
+        if (options_.max_image_size > 0) {
+            bitmaps_[image_idx]->Rescale((int)width, (int)height);
+        }
+
+        // Read and rescale depth map
+        depth_maps_[image_idx].reset(new DepthMap());
+        depth_maps_[image_idx]->Read(GetDepthMapPath(image_idx));
+        if (options_.max_image_size > 0) {
+            depth_maps_[image_idx]->Downsize(width, height);
+        }
+    }
+}

```

```

+ // Read and rescale confidence map
+ const std::string confidence_map_path = GetConfidenceMapPath(image_idx);
+ if (HasConfidenceMap(image_idx)) {
+     confidence_maps[image_idx].reset(new ConfidenceMap());
+     confidence_maps[image_idx]->Read(confidence_map_path);
+     if (options_.max_image_size > 0) {
+         confidence_maps[image_idx]->Downsize(width, height);
+     }
+ } else {
+     // Assume depth confidence probability of 1.0 when the map is not given
+     confidence_maps[image_idx].reset(new ConfidenceMap(width, height));
+     if (options_.save_calculated_maps) {
+         confidence_maps[image_idx]->Write(confidence_map_path);
+     }
+ }
+
+ // Read and rescale normal map
+ const std::string normal_map_path = GetNormalMapPath(image_idx);
+ normal_maps[image_idx].reset(new NormalMap());
+ if (!options_.calculate_normals && HasNormalMap(image_idx)) {
+     normal_maps[image_idx]->Read(normal_map_path);
+     if (options_.max_image_size > 0) {
+         normal_maps[image_idx]->Downsize(width, height);
+     }
+ } else {
+     // Estimate normal map from depth when the map is not given
+     normal_maps[image_idx]->EstimateFromDepth(
+         *depth_maps[image_idx], model_.images.at(image_idx).GetInvK());
+     if (options_.save_calculated_maps) {
+         normal_maps[image_idx]->Write(normal_map_path);
+     }
+ }
+ }
+
+ } // namespace mvs
+ } // namespace colmap

```

diff --git a/src/mvs/workspace.h b/src/mvs/workspace.h

index 60b2d16a..7afac240 100644

--- a/src/mvs/workspace.h

+++ b/src/mvs/workspace.h

@@ -42,6 +42,8 @@

namespace colmap {

namespace mvs {

```

+class ConfidenceMap;
+
+class Workspace {
+public:
+    struct Options {
@@ -68,9 +70,10 @@ class Workspace {
+        const Options& GetOptions() const;

+        const Model& GetModel() const;
-    const Bitmap& GetBitmap(const int image_idx);
-    const DepthMap& GetDepthMap(const int image_idx);
-    const NormalMap& GetNormalMap(const int image_idx);
+    virtual const Bitmap& GetBitmap(const int image_idx);
+    virtual const DepthMap& GetDepthMap(const int image_idx);
+    virtual const ConfidenceMap& GetConfidenceMap(const int image_idx);
+    virtual const NormalMap& GetNormalMap(const int image_idx);

+        // Get paths to bitmap, depth map, normal map and consistency graph.
+        std::string GetBitmapPath(const int image_idx) const;
@@ -82,7 +85,10 @@ class Workspace {
+        bool HasDepthMap(const int image_idx) const;
+        bool HasNormalMap(const int image_idx) const;

-    private:
+        // Do nothing when we use a cache. Data is loaded as needed.
+        virtual void Load(const std::vector<std::string>& image_names) {}
+
+    protected:
+        std::string GetFileName(const int image_idx) const;

+        class CachedImage {
@@ -107,6 +113,35 @@ class Workspace {
+        std::string normal_map_path_;
+    };

+class NoCacheWorkspace : public Workspace {
+public:
+    NoCacheWorkspace(const Options& options) : Workspace(options) {}
+
+    void Load(const std::vector<std::string>& image_names) override;
+
+    inline const Bitmap& GetBitmap(const int image_idx) override {
+        return *bitmaps_[image_idx];

```

```

+ }
+
+ inline const DepthMap& GetDepthMap(const int image_idx) override {
+     return *depth_maps_[image_idx];
+ }
+
+ inline const ConfidenceMap& GetConfidenceMap(const int image_idx) override {
+     return *confidence_maps_[image_idx];
+ }
+
+ inline const NormalMap& GetNormalMap(const int image_idx) override {
+     return *normal_maps_[image_idx];
+ }
+
+ private:
+     std::vector<std::unique_ptr<Bitmap>> bitmaps_;
+     std::vector<std::unique_ptr<DepthMap>> depth_maps_;
+     std::vector<std::unique_ptr<ConfidenceMap>> confidence_maps_;
+     std::vector<std::unique_ptr<NormalMap>> normal_maps_;
+);
+
+ // Import a PMVS workspace into the COLMAP workspace format. Only images in the
+ // provided option file name will be imported and used for reconstruction.
+ void ImportPMVSWorkspace(const Workspace& workspace,
+ diff --git a/src/util/option_manager.cc b/src/util/option_manager.cc
+ index 39c1b930..6ce3d1e0 100755
+ --- a/src/util/option_manager.cc
+ +++ b/src/util/option_manager.cc
+ @@ -591,6+591,8 @@ void OptionManager::AddPatchMatchStereoOptions() {
+     }
+     added_patch_match_stereo_options_ = true;
+
+ + AddAndRegisterDefaultOption("StereoFusion.mask_path",
+ +                             &stereo_fusion->mask_path);
+     AddAndRegisterDefaultOption("PatchMatchStereo.max_image_size",
+                                 &patch_match_stereo->max_image_size);
+     AddAndRegisterDefaultOption("PatchMatchStereo.gpu_index",
+ @@ -638,6+640,10 @@ void OptionManager::AddPatchMatchStereoOptions() {
+         &patch_match_stereo->filter_geom_consistency_max_cost);
+     AddAndRegisterDefaultOption("PatchMatchStereo.cache_size",
+                                 &patch_match_stereo->cache_size);
+ + AddAndRegisterDefaultOption("StereoFusion.use_cache",
+ +                             &stereo_fusion->use_cache);
+ + AddAndRegisterDefaultOption("StereoFusion.save_calculated_maps",

```

```
+ &stereo_fusion->save_calculated_maps);  
AddAndRegisterDefaultOption("PatchMatchStereo.write_consistency_graph",  
    &patch_match_stereo->write_consistency_graph);  
}
```

User Prompt:

You are a senior software engineer reviewing a pull request diff in the COLMAP repository. COLMAP is a general-purpose Structure-from-Motion and Multi-View Stereo pipeline written in C++ with CUDA support, used for 3D reconstruction from image collections. Your role is to verify that the changes correctly solve the stated problem, are production-ready, and don't introduce regressions or subtle bugs.

COLMAP's stereo fusion step is currently single-threaded and relies on an LRU cache to load workspace data (bitmaps, depth maps, normal maps) from disk on demand during the fusion loop. For large-scale reconstructions with hundreds or thousands of images, this sequential processing and repeated disk I/O is a major performance bottleneck. Users needed the fusion step to run significantly faster by utilizing multiple CPU cores without sacrificing output correctness.

This PR introduces parallelized stereo fusion by pre-loading the entire workspace into memory via a new NoCacheWorkspace class, then distributing the pixel-level fusion loop across threads using OpenMP. Per-thread intermediate data structures (fusion queues, accumulated coordinates, normals, colors, and visibility sets) are used to avoid data races. The PR also adds bounding box-filtering for fused points, per-image PNG mask support, a `use_cache_toggle` to select between cached and pre-loaded behavior, and a `save_calculated_maps` option. The expected result is that when `use_cache` is false (the new default), all workspace data is loaded in parallel at startup, and the fusion loop runs with `omp_get_max_threads()` threads, producing the same point cloud as the original single-threaded code but significantly faster.

A full unified diff is attached as a reference text. Review the diff as the sole source of code changes. Return a detailed PR review with issues, risks, and required changes before the merge.

Output format:

Summary

Blockers (for each, state what's wrong, why it matters, how to fix it, where it exists)

Non-blocking suggestions

Tests/Validation

Summary

Blockers (for each blocker, include what is wrong, why it matters, how to fix it and where it exists in the diff)

Non-blocking suggestions

Tests/Validation

Golden response:

1. Summary

This PR introduces parallelized stereo fusion, a `NoCacheWorkspace` for pre-loading all workspace data, bounding-box filtering, and per-image mask support. The architectural approach is sound: per-thread intermediate buffers avoid most data races during the pixel fusion loop, and `#pragma omp critical` protects the shared output vectors. However, the PR contains two show-stopping bugs that make it non-functional: (1) the `NoCacheWorkspace::Load` method has a for-loop that never increments its loop variable, causing an infinite hang, and (2) the workspace pointer is unconditionally overwritten with a plain `Workspace` immediately after the conditional `NoCacheWorkspace` `Workspace` creation, destroying the pre-loaded workspace. Additionally, there is a data race on `fused_pixel_masks_` in the parallel fusion loop, and several `StereoFusion` options are registered in the wrong option-manager function. These must be fixed before the merge.

2. Blockers

Blocker 1: Infinite loop in `NoCacheWorkspace::Load`, missing loop increment

- What's wrong: In `src/mys/workspace.cc`, the parallel for-loop in `NoCacheWorkspace::Load` is written as `for (int i = 0; i < image_names.size(); i)`. The loop increment expression is `i` (a no-op) rather than `++i` or `i++`. This means `i` is never incremented, and the loop body re-processes index 0 forever.
- Why it matters: This is an infinite loop. When `use_cache` is false (the new default path), calling `workspace->Load(image_names)` will hang the application indefinitely. The entire pre-loading feature is broken.
- Exact fix: Change `i)` to `++i)` in the for-loop header.

- Where: `src/mvs/workspace.cc`, line containing `for (int i = 0; i < image\_names.size(); i)`.

Blocker 2: Workspace pointer unconditionally overwritten, NoCacheWorkspace is always destroyed.

- What's wrong: In `src/mvs/fusion.cc` inside `StereoFusion::Run()`, the code first conditionally creates either a `Workspace` (when `use\_cache` is true) or a `NoCacheWorkspace` (when `use\_cache` is false) and calls `Load()` on the latter. However, immediately after this conditional block, the line `workspace\_.reset(new Workspace(workspace\_options));` unconditionally overwrites `workspace\_` with a plain `Workspace`, discarding the just-created `NoCacheWorkspace` and all of its pre-loaded data.

- Why it matters: This renders the entire `NoCacheWorkspace` code path dead. The preloaded bitmaps, depth maps, normal maps, and confidence maps are immediately freed. Subsequent fusion operations will use the default `Workspace` (cache-based), so the parallelization with `num\_threads = omp\_get\_max\_threads()` will hit the non-thread-safe LRU cache, causing data races and incorrect results. Effectively, the core feature of this PR does not work.

- Exact fix: Remove the unconditional `workspace\_.reset(new Workspace(workspace\_options));` line that appears after the `if/else` block.

- Where: `src/mvs/fusion.cc`, the line `workspace\_.reset(new Workspace(workspace\_options));` that appears after the `if (options\_.use\_cache)` / `else` block.

Blocker 3: Data race on fused_pixel_masks_ in the parallel fusion loop

- What's wrong: The `Fuse(tid)` method, called from within a `#pragma omp parallel for` region, reads and writes entries in `fused\_pixel\_masks\_` via `fused\_pixel\_mask.Get(row, col)` and `fused\_pixel\_mask.Set(row, col, true)`. The BFS-like traversal inside `Fuse()` follows overlapping images and accesses `fused\_pixel\_masks\_.at(image\_idx)` for image indices other than the currently-fused image. Since multiple threads are simultaneously fusing different pixels of the same image, two threads can traverse to the same neighboring image and concurrently read/write the same pixel mask entry without any synchronization.

- Why it matters: Concurrent unsynchronized reads and writes to the same `Mat<bool>` constitute undefined behavior in C++. In practice, this can cause duplicate fused points (two threads fuse the same pixel because both see it as

not-yet-fused), missed pixels, or memory corruption. The resulting point cloud may contain duplicates or be non-deterministic across runs.

- Exact fix: Either (a) protect ``fused_pixel_masks_`` access for cross-image pixels with an ``#pragma omp atomic`` or per-image mutex, or (b) restrict the parallel traversal so that ``Fuse()`` only marks pixels within the current image (the one being iterated in the outer loop) and defers cross-image pixel marking to a serial pass.

- Where: ``src/mvs/fusion.cc``, inside the ``Fuse(int tid)`` method, all accesses to ``fused_pixel_masks_.at(image_idx)`` where ``image_idx`` may differ from the image currently being processed in the parallel for-loop.

Blocker 4: StereoFusion options registered in `AddPatchMatchStereoOptions()` instead of `AddStereoFusionOptions()`

- What's wrong: In ``src/util/option_manager.cc``, the new options ``StereoFusion.mask_path``, ``StereoFusion.use_cache``, and ``StereoFusion.save_calculated_maps`` are registered inside the ``AddPatchMatchStereoOptions()`` method. These are StereoFusion options and should be registered in ``AddStereoFusionOptions()``.

- Why it matters: If a caller adds only StereoFusion options (e.g., via ``AddStereoFusionOptions()``), these three options will not be registered and will silently use their defaults. Users who try to set ``use_cache=true`` or provide a ``mask_path`` via the StereoFusion config will find their settings ignored unless they also happen to load PatchMatchStereo options.

- Exact fix: Move the three ``AddAndRegisterDefaultOption`` calls for ``StereoFusion.mask_path``, ``StereoFusion.use_cache``, and ``StereoFusion.save_calculated_maps`` from ``AddPatchMatchStereoOptions()`` to ``AddStereoFusionOptions()``.

- Where: ``src/util/option_manager.cc``, the lines adding ``StereoFusion.mask_path``, ``StereoFusion.use_cache``, and ``StereoFusion.save_calculated_maps``.

3. Non-blocking suggestions

- No validation of bbox file content: In ``src/exe/colmap.cc``, the bounding-box file is parsed using stream extraction (``file >> min_bound(0) >> ...``) with no check that all six values were successfully read. A malformed or truncated file will leave some bound components at their default (``-FLT_MAX`` or ``FLT_MAX``), producing silently incorrect filtering. Add a check on ``file.good()`` or ``file.fail()`` after reading.

- ``std::cout`` in parallel region: The warning message inside ``NoCacheWorkspace::Load``'s ``#pragma omp parallel for`` loop in ``src/mvs/workspace.cc`` uses ``std::cout`` without synchronization, which can produce interleaved output. Consider using ``#pragma omp critical`` around the warning output or collecting warnings for post-loop reporting.

4. Tests / Validation

1. Infinite loop regression test: Verify that ``NoCacheWorkspace::Load`` completes in finite time for a workspace with at least 2 images. Before the fix, this hangs; after the fix, it returns normally.

2. Workspace type preservation test: After ``StereoFusion::Run()`` initializes the workspace with ``use_cache = false``, verify that ``workspace_`` is an instance of ``NoCacheWorkspace`` (e.g., via ``dynamic_cast``) and not a plain ``Workspace``. Before the fix, this fails; after the fix, it succeeds.

3. Deterministic output test: Run stereo fusion on a small dataset (e.g., 3-5 images) with multiple threads and compare the output point cloud against a single-threaded reference run. The fused point count should be identical, and point positions should match within floating-point tolerance. This validates the data race fix on ``fused_pixel_masks_``.

4. Option registration test: Set ``StereoFusion.use_cache=true`` and ``StereoFusion.mask_path=/some/path`` via the fusion config file, using only ``AddStereoFusionOptions()``. Verify the options are correctly parsed and applied. Before the fix (options in the wrong function), they are ignored; after the fix, they take effect.

Rubric:

Criteria	Rubric Type	Rubric Weights
		Mandatory = 5

		Important = 3 Nice to Have = 1
The response follows a structure with Summary, Blockers, Non-blocking Suggestions, and Tests/Validations sections.	Instruction following	mandatory
The response identifies that the for-loop in <code>NoCacheWorkspace::Load</code> in <code>src/mvs/workspace.cc</code> has a missing increment (<code>i</code> instead of <code>++i</code> or <code>i++</code>), causing an infinite loop.	bug discovery	mandatory
The response suggests a fix for the infinite loop in <code>NoCacheWorkspace::Load</code> , for example, changing the loop increment from <code>i</code> to <code>++i</code> or <code>i++</code> in the for-loop header.	bug discovery	mandatory
The response identifies that in <code>StereoFusion::Run()</code> in <code>src/mvs/fusion.cc</code> , the line <code>workspace_.reset(new Workspace(workspace_options))</code> unconditionally overwrites the workspace pointer after the conditional <code>NoCacheWorkspace</code> / <code>Workspace</code> creation block, destroying the pre-load <code>NoCacheWorkspace</code> and its data.	bug discovery	mandatory
The response suggests a fix for the unconditional workspace overwrite, for example, removing or guarding the <code>workspace_.reset(new Workspace(workspace_options))</code> line so that the <code>NoCacheWorkspace</code> created in the else-branch is preserved	bug discovery	mandatory
The response identifies that there is a data race on <code>fused_pixel_masks_</code> when <code>Fuse()</code> is called from multiple threads in the OpenMp parallel region, because the BFS traversal can read/write pixel masks for overlapping images concurrently without synchronization	bug discovery	mandatory

The response suggests a fix for the `fused_pixel_masks` data race, for example, adding per-image mutexes, using atomic operations on mask entries, or restricting cross-image mask writes to a serial phase	bug discovery	mandatory
The response identifies that the `StereoFusion.mask_path`, `StereoFusion.use_cache`, and `StereoFusion.save_calculated_maps` options are registered inside `AddPatchMatchStereoOptions()` in `src/util/option_manager.cc` instead of `AddStereoFusionOptions()`, meaning they will not be available when only stereo fusion options are loaded. (Note: This rubric can be split into three)	bug discovery	mandatory
The response suggests a fix for the misplaced option registrations, for example, moving the three `StereoFusion.mask_path`, `StereoFusion.use_cache`, and `StereoFusion.save_calculated_maps` option registrations from `AddPatchMatchStereoOptions()` to `AddStereoFusionOptions()` (Note: This rubric can be split into three)	bug discovery	mandatory
The response includes a non-blocking suggestion that the bounding-box file parsing in `src/exe/colmap.cc` does not validate that all six float values were successfully read, risking silently incorrect bounds from a malformed file.	Logic Reasoning	important
The response includes a non-blocking suggestion that the warning message printed via `std::cout` inside the `pragma omp parallel for` loop in `NoCacheWorkspace::Load` in `src/mvs/workspace.cc` lacks synchronization, which can produce interleaved output when multiple threads print simultaneously.	Logic Reasoning	important
The response recommends a test that verifies `NoCacheWorkspace::Load` completes without hanging, for	instruction following	important

example, by running it on a workspace with at least 2 images and confirming it returns in a finite time		
The response recommends a test that verifies the workspace pointer retains the `NoCacheWorkspace` type when `use_cache` is false, for example, by checking via `dynamic_cast` after initialization.	instruction following	important

 Example #3

None

Task Metadata:

Github Repo

<https://github.com/wandb/wandb>

Repository Name and Description

wandb/wandb

The AI developer platform. Use Weights & Biases to train and fine-tune models, and manage models from experimentation to production. - wandb/wandb

Base Commit

f2f7eecf2bec31ab89cb122714c9cef47f5e2bf5

Target Commit

ef20cb95d96ea83e43dbabac9a0ee9d741a0b395

PR Link

<https://github.com/wandb/wandb/pull/10953>

Diff:

```
diff --git a/CHANGELOG.unreleased.md b/CHANGELOG.unreleased.md
index e649faed1..6578e3480 100644
--- a/CHANGELOG.unreleased.md
+++ b/CHANGELOG.unreleased.md
```

@@ -16,6 +16,7 @@ Section headings should be at level 3 (e.g. `### Added`).

Added

- The automations API now supports creating and editing automations that trigger on run states (@tonyyl-wandb in <https://github.com/wandb/wandb/pull/10848>)

+ The automations API now support basic zscore automation events (@matthoore117-wandb in <https://github.com/wandb/wandb/pull/10931>)

Deprecated

diff --git a/core/api/graphql/schemas/commit.hash.txt b/core/api/graphql/schemas/commit.hash.txt
index a8aebd5e1..532c227be 100644

--- a/core/api/graphql/schemas/commit.hash.txt

+++ b/core/api/graphql/schemas/commit.hash.txt

@@ -1,1 +1,1 @@

-2a7440dd915565678a4a073310c8fcb7cb0a6f11

+dd114cf9af2633d6d39de0adb068cb19b5dcd045

diff --git a/wandb/automations/__init__.py b/wandb/automations/__init__.py

index b3e9f1be0..b9d8738c6 100644

--- a/wandb/automations/__init__.py

+++ b/wandb/automations/__init__.py

@@ -8,6 +8,7 @@ from .events import (

EventType,

MetricChangeFilter,

MetricThresholdFilter,

+ MetricZScoreFilter,

OnAddArtifactAlias,

OnCreateArtifact,

OnLinkArtifact,

@@ -62,6 +63,7 @@ __all__ = [

"MetricThresholdFilter",

"MetricChangeFilter",

"RunStateFilter"

+ "MetricZScoreFilter",

Actions

"ActionType", # doc:exclude

"SendNotification",

diff --git a/wandb/automations/_filters/run_metrics.py b/wandb/automations/_filters/run_metrics.py

index 3f5906f2b..920f5545d 100644

--- a/wandb/automations/_filters/run_metrics.py

+++ b/wandb/automations/_filters/run_metrics.py

@@ -332,6 +332,9 @@ class MetricVal(BaseMetricOperand):

def mean(self, window: int) -> MetricAgg:

return self.avg(window=window)

```

+ def zscore(self, window: int) -> MetricZScoreFilter:
+     return MetricZScoreFilter(name=self.name, window=window)
+
+
+ class MetricAgg(BaseMetricOperand):
+     """Represents an aggregated metric value when defining metric event filters."""
+     @@ -339,3 +342,87 @@ class MetricAgg(BaseMetricOperand):
+         name: str
+         agg: Annotated[Agg, Field(alias="agg_op")]
+         window: Annotated[PositiveInt, Field(alias="window_size")]
+
+
+
+ class MetricZScoreFilter(GQLBase, extra="forbid"):
+     name: str
+     """Name of the observed metric."""
+
+     window: Annotated[PositiveInt, Field(alias="window_size")] = 30
+     """Size of the window to calculate the metric mean and standard deviation over."""
+
+     threshold: PosNum = 3.0
+     """Threshold for the z-score."""
+
+     change_dir: ChangeDir = ChangeDir.ANY
+     """Direction of the z-score change to watch for."""
+
+     __change_dir_abs_set: bool = False
+     """Flag to track if the change_dir has been set to ABSOLUTE."""
+
+     def __and__(self, other: Any) -> RunMetricFilter:
+         """Returns `(metric_filter & run_filter)` as a `RunMetricFilter`."""
+         from wandb.automations.events import RunMetricFilter
+
+         if isinstance(run_filter := other, (BaseOp, FilterExpr)):
+             # Treat `other` as a run filter and build a RunMetricEvent. Let the
+             # metric filter validators wrap or nest as appropriate.
+             return RunMetricFilter(run=run_filter, metric=self)
+         return NotImplemented
+
+     def __rand__(self, other: BaseOp | FilterExpr) -> RunMetricFilter:
+         """Ensures `&` is commutative for run and metric filters.
+
+         I.e. `(run_filter & metric_filter) == (metric_filter & run_filter)`.
+
+         """
+         return self.__and__(other)

```

```

+
+ def __repr__(self) -> str:
+     if self.change_dir is ChangeDir.ANY:
+         return repr(rf"abs(zscore({self.name!r})) > {self.threshold}")
+     elif self.change_dir is ChangeDir.DECREASE:
+         return repr(rf"zscore({self.name!r}) < -{self.threshold}")
+     else: # ChangeDir.INCREASE
+         return repr(rf"zscore({self.name!r}) > +{self.threshold}")
+
+ @override
+ def __rich_repr__(self) -> RichReprResult:
+     """Returns the `rich` pretty-print representation of the metric filter."""
+     # See: https://rich.readthedocs.io/en/stable/pretty.html#rich-repr-protocol
+     yield None, repr(self)
+
+ def gt(self, value: int | float, /) -> MetricZScoreFilter:
+     """Returns a filter that watches for `abs(zscore(metric_expr)) > threshold`."""
+     self.change_dir = (
+         self.change_dir if self.__change_dir_abs_set else ChangeDir.INCREASE
+     )
+     if value < 0:
+         raise ValueError(f"Expected positive threshold, got: {value=}")
+     self.threshold = value
+     return self
+
+ def lt(self, value: int | float, /) -> MetricZScoreFilter:
+     """Returns a filter that watches for `abs(zscore(metric_expr)) < threshold`."""
+     self.change_dir = (
+         self.change_dir if self.__change_dir_abs_set else ChangeDir.DECREASE
+     )
+     if value < 0:
+         raise ValueError(f"Expected positive threshold, got: {value=}")
+     self.threshold = value
+     return self
+
+ def abs(self) -> MetricZScoreFilter:
+     """Returns a filter that watches for `abs(zscore(metric_expr)) > threshold`."""
+     self.change_dir = ChangeDir.ANY
+     self.__change_dir_abs_set = True
+     return self
+
+ def __gt__(self, other: Any) -> MetricZScoreFilter:
+     if isinstance(other, (int, float)):
+         return self.gt(other)

```

```

+         return NotImplemented
+
+     def __lt__(self, other: Any) -> MetricZScoreFilter:
+         if isinstance(other, (int, float)):
+             return self.lt(other)
+         return NotImplemented
diff --git a/wandb/automations/_generated/enums.py b/wandb/automations/_generated/enums.py
index e8e4ef7df..e28838f27 100644
--- a/wandb/automations/_generated/enums.py
+++ b/wandb/automations/_generated/enums.py
@@ -26,6 +26,7 @@ class EventTriggeringConditionType(str, Enum):
     RUN_METRIC = "RUN_METRIC"
     RUN_METRIC_CHANGE = "RUN_METRIC_CHANGE"
     RUN_STATE = "RUN_STATE"
+    RUN_METRIC_ZSCORE = "RUN_METRIC_ZSCORE"

class TriggeredActionType(str, Enum):
diff --git a/wandb/automations/events.py b/wandb/automations/events.py
index 370f39249..c47ba9c13 100644
--- a/wandb/automations/events.py
+++ b/wandb/automations/events.py
@@ -12,7 +12,12 @@ from wandb._strutils import nameof

from ._filters import And, MongoLikeFilter
from ._filters.expressions import FilterableField
-from ._filters.run_metrics import MetricChangeFilter, MetricThresholdFilter, MetricVal
+from ._filters.run_metrics import (
+    MetricChangeFilter,
+    MetricThresholdFilter,
+    MetricVal,
+    MetricZScoreFilter,
+)
from ._filters.run_states import StateFilter, StateOperand
from ._generated import FilterEventFields
from ._validators import (
@@ -48,6 +53,7 @@ class EventType(LenientStrEnum):
     RUN_METRIC_THRESHOLD = "RUN_METRIC"
     RUN_METRIC_CHANGE = "RUN_METRIC_CHANGE"
     RUN_STATE = "RUN_STATE"
+    RUN_METRIC_ZSCORE = "RUN_METRIC_ZSCORE"

# -----

```

```
@@ -94,6 +100,22 @@ class _WrappedMetricChangeFilter(GQLBase): # from: RunMetricFilter
    return v
```

```
+class _WrappedMetricZScoreFilter(GQLBase): # from: RunMetricFilter
```

```
+    event_type: Annotated[
+        Literal[EventType.RUN_METRIC_ZSCORE],
+        Field(exclude=True, repr=False),
+    ] = EventType.RUN_METRIC_ZSCORE
```

```
+    zscore_filter: MetricZScoreFilter
```

```
+    @model_validator(mode="before")
```

```
+    @classmethod
```

```
+    def _nest_inner_filter(cls, v: Any) -> Any:
```

```
+        if pydantic_isinstance(v, MetricZScoreFilter):
```

```
+            return cls(zscore_filter=v)
```

```
+        return v
```

```
+    +
```

```
class RunMetricFilter(GQLBase): # from: TriggeringRunMetricEvent
```

```
    run: Annotated[
```

```
        JsonEncoded[MongoLikeFilter],
```

```
@@ -103,7 +125,11 @@ class RunMetricFilter(GQLBase): # from: TriggeringRunMetricEvent
```

```
    """Filters that must match any runs that will trigger this event."""
```

```
    metric: Annotated[
```

```
-        Union[_WrappedMetricThresholdFilter, _WrappedMetricChangeFilter],
```

```
+        Union[
```

```
+            _WrappedMetricThresholdFilter,
```

```
+            _WrappedMetricChangeFilter,
```

```
+            _WrappedMetricZScoreFilter,
```

```
+        ],
```

```
        Field(alias="run_metric_filter"),
```

```
    ]
```

```
    """Metric condition(s) that must be satisfied for this event to trigger."""
```

```
@@ -123,7 +149,9 @@ class RunMetricFilter(GQLBase): # from: TriggeringRunMetricEvent
```

```
    def _nest_metric_filter(cls, v: Any) -> Any:
```

```
        # If no run filter is given, automatically nest the metric filter and
```

```
        # let inner validators reshape further as needed.
```

```
-        if pydantic_isinstance(v, (MetricThresholdFilter, MetricChangeFilter)):
```

```
+        if pydantic_isinstance(v,
```

```
+            (MetricThresholdFilter, MetricChangeFilter, MetricZScoreFilter)
```

```
+        ):
```

```

        return cls(metric=v)
    return v

@@ -301,7 +329,11 @@ class OnRunMetric(_BaseRunEventInput):
    ...

    """

-   event_type: Literal[EventType.RUN_METRIC_THRESHOLD, EventType.RUN_METRIC_CHANGE]
+   event_type: Literal[
+       EventType.RUN_METRIC_THRESHOLD,
+       EventType.RUN_METRIC_CHANGE,
+       EventType.RUN_METRIC_ZSCORE,
+   ]

    filter: JsonEncoded[RunMetricFilter]
    """Run and/or metric condition(s) that must be satisfied for this event to trigger."""
@@ -401,4 +433,5 @@ __all__ = [
    "ArtifactEvent",
    "MetricThresholdFilter",
    "MetricChangeFilter",
+   "MetricZScoreFilter",
]

```

User Prompt:

You are a senior software engineer reviewing a pull request diff for the Weights and Biases SDK (wandb/wandb), an open-source ML developer platform used for experiment tracking, model training, and production ML workflows. Your role is to verify that the changes correctly solve the stated problem, are production-ready, and do not introduce bugs. Pay close attention to API design consistency with existing patterns, correctness of operator semantics, and Pydantic model behavior.

The goal is to extend the automations API so users can create automations that trigger based on z-score deviations of run metrics. The changes add a new `MetricZScoreFilter` class with comparison operators (`gt`, `lt`, `abs`), a `_WrappedMetricZScoreFilter` wrapper for GraphQL event input. The implementation follows the existing patterns established by `MetricThresholdFilter` and `MetricChangeFilter`.

A full unified diff is attached as reference text. Review the diff as the sole source of code changes. Return a detailed PR review with issues, risks, and required changes before merge with the following Output format:

Summary

Blockers (for each blocker, include what is wrong, why it matters, how to fix it and where it exists in the diff)

Non-blocking suggestions

Tests/Validation

Golden response:

1. Summary

This PR adds z-score-based metric filtering to the wandb automations API, allowing users to trigger automations when a run metric's z-score exceeds a threshold. The implementation follows the existing patterns for `MetricThresholdFilter` and `MetricChangeFilter`, adding a new `MetricZScoreFilter` class with comparison operators (`gt`, `lt`, `abs`), a `WrappedMetricZScoreFilter` wrapper for GraphQL serialization, and wiring the new filter type into the existing `RunMetricFilter` union, `EventType` enum, and `OnRunMetric` event input. While the overall structure and GraphQL integration are correct, there are two must fix issues: the `gt()` and `lt()` methods mutate the filter object in place which breaks operator chaining semantics, and the threshold validation allows zero which contradicts the `PosNum` type.

2. Blockers

Blocker 1: `gt()` and `lt()` mutate self in place instead of returning new instances

What's wrong: Both `gt()` and `lt()` modify `self.change_dir` and `self.threshold` directly, then return `self`. This means calling `filter.gt(2)` permanently alters the original filter object. If a user stores a base filter and reuses it to create variants (e.g., `base = metric.zscore(30)`; `up = base.gt(2)`; `down = base.lt(3)`), `down` will overwrite the threshold and direction already set by `up`, and both `up` and `down` will reference the same mutated object. The same problem affects `abs()`.

Why it matters: This is a correctness bug that produces silent wrong behavior. Users building multiple automations from a shared base filter will get

incorrect triggers. A fluent builder API which this class is clearly designed to be (given the method chaining pattern) should return new, independent instances from each call so that callers can safely branch from a common base configuration.

How to fix: Have `gt()`, `lt()`, and `abs()` return new `MetricZScoreFilter` instances (e.g., via `self.model_copy(update={...})`) instead of mutating and returning `self`.

Where: `wandb/automations/_filters/run_metrics.py`, the `gt()`, `lt()`, and `abs()` methods of `MetricZScoreFilter`.

Blocker 2: Threshold validation allows zero, contradicting `PosNum` type

What's wrong: The `gt()` and `lt()` methods validate if `value < 0`: raise `ValueError(...)`, which means `value=0` passes the manual guard and gets assigned to `self.threshold`. However, the threshold field is declared as `PosNum`, which by definition only accepts strictly positive numbers. The manual validation is therefore inconsistent with the declared type, it raises for negatives but silently accepts zero, which `PosNum` should reject.

Why it matters: Users calling `.gt(0)` or `.lt(0)` will not get an error at the point of filter construction, but will end up with a threshold of zero that makes no mathematical sense for z-score filtering. A z-score of zero means no deviation from the mean, so the filter would trigger constantly, producing noisy and useless automation events.

How to fix: Change the validation check from `if value < 0` to `if value <= 0` to reject zero, matching the `PosNum` semantics.

Where: `wandb/automations/_filters/run_metrics.py`, the validation check in both `gt()` and `lt()`.

3. Non-blocking Suggestions

`gt()` docstring is incorrect: The `gt()` method's docstring says "Returns a filter that watches for `abs(zscore(metric_expr)) > threshold`", but the implementation sets `change_dir = ChangeDir.INCREASE` (not `ChangeDir.ANY`), producing `zscore(metric_expr) > +threshold`, a directional check, not an absolute value check. To fix, change the docstring to "Returns a filter that watches for `zscore(metric_expr) > threshold`".

lt() docstring is incorrect: The lt() method's docstring says "Returns a filter that watches for `abs(zscore(metric_expr)) < threshold`", but lt() sets `change_dir = ChangeDir.DECREASE`, meaning it watches for `zscore(metric) < -threshold`. To fix, change the docstring to "Returns a filter that watches for `zscore(metric_expr) < -threshold`".

CHANGELOG grammar: The added line reads "now support basic zscore": it should be "now supports" (third person singular) to match the existing entry above it ("now supports creating and editing").

`__all__` ordering in `__init__.py`: `MetricZScoreFilter` is appended after `RunStateFilter` in `__all__`, breaking the grouping where other metric filters are listed together (`MetricThresholdFilter`, `MetricChangeFilter`). Consider placing it adjacent to the other metric filters for consistency.

Undocumented default window size: `MetricZScoreFilter` defaults window to 30 with no explanation of what this value represents (30 data points? 30 log steps? 30 seconds?). Consider adding a docstring or comment explaining rationale for this default.

4. Tests/validation

The following tests should be written before merge:

- Operator independence test: Create a base filter `base = metric.zscore(30)`, then derive `up = base.gt(2)` and `down = base.lt(3)`. Assert that `up.threshold` is 2, `down.threshold` is 3, `up.change_dir` is `INCREASE`, `down.change_dir` is `DECREASE`, and `base` is unmodified. Before the fix (Blocker 1), `up` and `down` will be the same mutated object; after the fix, they will be independent instances.

- Zero threshold rejection: Call `metric.zscore(30).gt(0)` and assert it raises `ValueError`. Before the fix (Blocker 2), this silently succeeds with `threshold=0`.

- `RunMetricFilter` union discrimination: Create a `RunMetricFilter` with a `MetricZScoreFilter` as the metric and assert that the resulting metric field is an instance of `_WrappedMetricZScoreFilter`, confirming that the discriminated union correctly selects the z-score wrapper.

Rubric:

Criteria	Rubric Type	Rubric Weights Mandatory = 5 Important = 3 Nice to Have = 1
The response follows a structure with Summary, Blockers, Non-blocking Suggestions and Tests/Validations sections.	Instruction following	mandatory
The response identifies that the <code>gt()</code> , <code>lt()</code> , and <code>abs()</code> methods in <code>MetricZScoreFilter</code> in <code>wandb/automations/_filters/run_metrics.py</code> mutate the filter object in place and returns <code>self</code> , creating a silent wrong behavior when reusing a base filter to derive multiple variants (e.g. calling <code>base.gt(2)</code> then <code>base.lt(3)</code> overwrites the first result since both <code>up</code> and <code>down</code> reference the same mutated object).	bug discovery	mandatory
The response suggests a fix for the in-place mutation problem in <code>gt()</code> , <code>lt()</code> , and <code>abs()</code> in <code>wandb/automations/_filters/run_metrics.py</code> , for example having each method return a new <code>MetricZScoreFilter</code> instance (e.g. via <code>self.model_copy(update={...})</code>) instead of mutating and returning <code>self</code> .	bug discovery	mandatory
The response identifies that the threshold validation in <code>gt()</code> and <code>lt</code> in <code>wandb/automations/_filters/run_metrics.py</code> uses <code>if value < 0</code> which allows zero to pass, despite the <code>threshold</code> field being	bug discovery	mandatory

declared as `PosNum` (strictly positive), making the manual guard inconsistent with the declared type.		
The response suggests a fix for the zero-threshold validation gap in `gt()` and `lt()` in `wandb/automations/_filters/run_metrics.py`, for example changing the check from `if value < 0` to `if value <= 0` to reject zero and align with the `PosNum` type declaration.	bug discovery	mandatory
The response includes a non-blocking suggestion that the `gt()` method's docstring in `wandb/automations/_filters/run_metrics.py` is incorrect because it says it watches for `abs(zscore(metric_expr)) > threshold` but the implementation sets `change_dir = ChangeDir.INCREASE`, meaning it actually watches for positive directional deviations only.	Style and Tone	nice to have
The response includes a non-blocking suggestion that the `lt()` method's docstring in `wandb/automations/_filters/run_metrics.py` is incorrect because it says it watches for `abs(zscore(metric_expr)) < threshold` but the method sets `change_dir = ChangeDir.DECREASE`, meaning it actually watches for negative deviations (`zscore < -threshold`), not for small absolute z-scores.	Style and Tone	nice to have
The response includes a non-blocking suggestion that the CHANGELOG entry added in `CHANGELOG.unreleased.md` contains a grammar error as it writes "now support basic zscore", which should instead be "now supports basic zscore" (missing 's' in supports).	Style and Tone	nice to have
The response includes a non-blocking suggestion that `MetricZScoreFilter` is appended after `RunStateFilter` in `__all__` in `wandb/automations/__init__.py` which breaks the grouping where the other metric filters (`MetricThresholdFilter`, `MetricChangeFilter`) are listed together.	Style and Tone	nice to have

The response includes a non-blocking suggestion that the `window` parameter in `MetricZScoreFilter` in `wandb/automations/_filters/run_metrics.py` has an undocumented default of `30`, mentioning that there is no explanation of what this value represents (e.g. 30 data points, 30 log steps, 30 seconds).	Style and Tone	nice to have
The response recommends a test that creates a base `MetricZScoreFilter` (e.g. via `metric.zscore(30)`), derives multiple variants via `gt()` and `lt()`, and asserts each variant has independent `threshold` and `change_dir` values and the original base filter is unmodified.	Instruction Following	important
The response recommends a test that verifies calling `gt(0)` or `lt(0)` on a `MetricZScoreFilter` raises a `ValueError`.	instruction following	important
The response recommends a test creates a `RunMetricFilter` with a `MetricZScoreFilter` as the metric and verifies that the resulting metric field is an instance of `WrappedMetricZScoreFilter`.	instruction following	important

Appendix

WIP